Quint + Claude + C++: Model-Based Testing of an Elevator Group

> A worked example of AI-assisted formal verification: from a plain-English design doc, to a model-checked Quint spec, to a Claude-generated C++ implementation, to trace-driven correctness testing with an automated fix loop.

Repository: https://github.com/Washoutx/quint_test

Table of Contents

1. The Example: Elevator Group Control
2. Phase 1 — The Design Docs
3. Phase 2 — Claude Generates the Quint Spec
4. The Quint Spec: Actions and Invariants Explained
5. Phase 3 — Quint Model-Checks the Spec
6. Phase 4 — Claude Generates the C++ Implementation
7. Phase 5 — Quint Emits ITF Traces
8. Phase 6 — The C++ MBT Driver
9. Phase 7 — The Fix Loop: Catching a Real Bug
10. Running the Whole Loop: Real Output
11. Appendix — Full Quint Spec (elevator.qnt)

---

1. The Example: Elevator Group Control

Instead of a toy data structure we model a small distributed system with real concurrency:

- **A building with 4 floors and 2 elevators** ("A" and "B"). Hall buttons (Up/Down) on every floor, a floor panel in every car.
- **Hall panels** light a lamp and send a hall call to the dispatcher.
- **The dispatcher** assigns each call to exactly one elevator, the closest one according to its *possibly stale* knowledge of the elevators' positions.
- **Elevator controllers** move floor by floor with the SCAN algorithm, open doors at target floors, and report position (with a sequence number) and served calls back.
- **An asynchronous network**: messages may be delivered in any order and after any delay, never lost.

This is exactly the kind of system where hand-written tests miss bugs: reordered status reports, assignments in flight while a car is moving, two cars competing for the same call. Random simulation explores those interleavings for us.

The workflow: a design doc is turned by Claude into a Quint spec; Quint checks the spec; Claude derives C++ from the verified spec; Quint generates random executions (ITF traces); a C++ driver replays every

trace against the implementation and reports the first divergence; Claude fixes the C++ until the loop passes. **The spec is the oracle**: nobody guesses what correct behaviour is.

---

2. Phase 1 — The Design Docs

Two documents drive the whole process. Both are written as prompts for Claude.

`docs/DESIGN_SPEC.md` describes the behaviour in plain English, with no formulas and no mention of any implementation language. Pasting it into Claude must produce `spec/elevator.qnt` and nothing else. It fixes the exact names of types, variables, actions, invariants and tests, because the C++ side and the traces depend on them.

docs/DESIGN_SPEC.md

Elevator Group Control Design

Overview

A building with four floors is served by two elevators, called "A" and "B". Every floor has hall buttons for going up and going down, and every car has a panel with one button per floor.

- **Hall panels**: when a passenger presses a hall button, its lamp lights up and a hall call is sent to the dispatcher.
- **The dispatcher** collects hall calls and gives each of them to exactly one elevator: the one that is closest to the calling floor, judged by the last position the dispatcher has heard from each elevator. That knowledge may be out of date.
- **Elevator controllers**: every car moves floor by floor using the SCAN algorithm (keep going in the same direction while there is work ahead, then turn around), opens its doors at the floors it has to serve, and reports its position and the calls it has served back to the dispatcher.

The dispatcher and the elevators talk only through an asynchronous network. Messages can arrive in any order and after any delay, but they are never lost and never duplicated. Car buttons are handled by the car itself and never go through the dispatcher.

Out of scope: passengers as individuals, load, real time (speed, door timers), failures of any part, moving a call to another elevator once it has been given out, and the direction of a hall call when serving it (a car that stops at a floor serves all of its hall calls there).

Types

- `Dir`: `Up` or `Down`.
- `Heading`: either `Going` with a `Dir`, or `Idle`. It is the direction the car is committed to. It is chosen when the car departs and goes back to `Idle` when the car has no more work. An `Idle` car may already have new work that arrived while it was standing still.
- `Door`: `Open` or `Closed`.
- `Motor`: `Stopped` or `Running`.
- `HallCall`: a record with a `floor` and a `dir`, meaning "someone on this floor wants to go in this direction".
- `ElevatorState`: a record describing one car:
  - `floor`: the floor the car is at; while moving, the last floor it reached,
  - `door`: whether the door is open or closed,
  - `motor`: whether the car is moving,

    - `heading`: the direction the car is committed to,
    - `carCalls`: the floors selected inside the car that have not been served,
    - `hallStops`: the hall calls given to this car that have not been served,
    - `statusSeq`: the number of the last position report this car has sent.
- `PositionReport`: a record with a `floor` and a `seq`: where the dispatcher believes an elevator is, and which report that belief came from.
- `DispatcherState`: a record with
    - `pending`: hall calls received but not yet given to any elevator,
    - `assigned`: a map from a hall call to the name of the elevator it was given to, kept until the elevator confirms the call was served,
    - `known`: a map from every elevator name to its last known position report.
- `Message`: one of four kinds:
    - `HallCallMsg` carries a `HallCall`, from a hall panel to the dispatcher,
    - `AssignMsg` carries an `elevator` name and a `call`, from the dispatcher to that elevator,
    - `StatusMsg` carries an `elevator` name, a `floor` and a `seq`, from an elevator to the dispatcher,
    - `ServedMsg` carries an `elevator` name and a `call`, from an elevator to the dispatcher.

Constants

- `NUM_FLOORS` is 4.
- `BOTTOM` is floor 0 and `TOP` is the highest floor, floor 3.
- `FLOORS` are all floors from the bottom to the top.
- `ELEVATORS` are the names "A" and "B".

State Variables

- `elevators`: the state of every car, by name.
- `dispatcher`: the dispatcher's state.
- `hallLamps`: the hall button lamps that are currently lit.
- `network`: the messages currently in flight. Sending puts a message into it, delivering takes one out. Any message in it may be delivered next.

Initial state

- Both cars stand on the bottom floor with the door closed, the motor stopped, heading `Idle`, no car calls, no hall stops, and report number 0.
- The dispatcher has nothing pending and nothing assigned, and believes both elevators are on the bottom floor, with report number 0.
- No hall lamp is lit and the network is empty.

Helpers

- `abs`: the absolute value of a number.
- `isValidHallCall`: a hall call is valid when its floor exists, it is not an up call on the top floor, and it is not a down call on the bottom floor.
- `HALL_CALLS`: all valid hall calls.
- `targets`: the floors a car has to stop at: all its car calls together with the floors of all its hall stops.
- `delta`: how a heading moves the car: one floor up for going up, one floor down for going down, nothing for idle.
- `nextDir` (SCAN): the direction a car should take next.
    1. If it is going up and has a target above it, it goes up.
    2. Otherwise, if it is going down and has a target below it, it goes down.

3. Otherwise, if it has any target above it, it goes up.
4. Otherwise it goes down.

- `closestElevators`: takes the dispatcher state and a floor, and returns the elevators whose position, as the dispatcher knows it, is nearest to that floor. When several are equally near, all of them are returned. Only the dispatcher's knowledge is used, never the real positions of the cars.
- `without`: a map of assigned calls with one call removed.

Every operation below leaves unchanged everything it does not mention.

The receive operations take the content carried by a message, not the message itself: `receiveHallCall` takes the hall call, and `receiveStatus`, `receiveServed` and `receiveAssign` take the record carried by their kind of message. The message they look for in the network is that content wrapped in the matching kind of message.

Operations

Passengers

pressHallButton(c)

- **Precondition**: `c` is a valid hall call and its lamp is not lit. Pressing a button that is already lit is therefore not possible.
- **Effect**: the lamp of `c` lights up and a `HallCallMsg` for `c` is sent.

pressCarButton(e, f)

- **Precondition**: floor `f` exists, the car `e` is not on that floor, and `f` is not already selected in that car. This is allowed while the car moves.
- **Effect**: `f` is added to the car calls of `e`.

Dispatcher

receiveHallCall(c)

- **Precondition**: a `HallCallMsg` for `c` is in the network.
- **Effect**: the message is taken out of the network. If the dispatcher has neither pending nor assigned `c` already, `c` becomes pending. Otherwise nothing else changes.

assignTo(c, e)

- **Precondition**: `c` is pending and `e` is one of the closest elevators to the floor of `c`.
- **Effect**: `c` is no longer pending, it is recorded as assigned to `e`, and an `AssignMsg` telling `e` to serve `c` is sent.

assignCall(c)

- **Effect**: one of the closest elevators to the floor of `c` is chosen non deterministically, this choice is named `target`, and `assignTo(c, target)` is performed.

receiveStatus(s)

- **Precondition**: the `StatusMsg` `s` is in the network.
- **Effect**: the message is taken out of the network. If its report number is higher than the one the dispatcher knows for that elevator, the dispatcher now believes the elevator is on the reported floor, with the new report number. Otherwise the report is outdated and is ignored.

receiveServed(s)

- **Precondition**: the `ServedMsg` `s` is in the network.
- **Effect**: the message is taken out of the network, the call is no longer assigned, and its hall lamp goes off. The lamp goes off only now, not when the doors open; until then, pressing the same hall button again has no effect.

Elevator controller

receiveAssign(a)

- **Precondition**: the `AssignMsg` `a` is in the network.
- **Effect**: the message is taken out of the network and the call is added to the hall stops of the elevator named in it.

departure(e)

- **Precondition**: the car `e` is stopped, its door is closed, it has at least one target, and its current floor is not a target. Work on the current floor has to be done first, by opening the doors.
- **Effect**: the motor starts and the heading becomes going in the direction chosen by `nextDir`.

travel(e)

- **Precondition**: the car `e` is running.
- **Effect**:
    - the car moves one floor in the direction of its heading,
    - if the new floor is one of its targets (as they were before this move), the motor stops; otherwise the car keeps running,
    - its report number goes up by one,
    - a `StatusMsg` with the car's name, the new floor and the new report number is sent to the dispatcher.
- There is deliberately no check that the new floor exists. That a car never leaves the building follows from the rest of the rules and is checked by INV-02.

openDoor(e)

- **Precondition**: the car `e` is stopped, its door is closed, and its current floor is one of its targets.
- **Effect**:
    - the car call for the current floor is removed,
    - all hall stops of the car on the current floor are served and removed,
    - the door opens,
    - if the car has no targets left, its heading becomes `Idle`; otherwise the heading stays as it was,
    - a `ServedMsg` is sent for every hall call that was served.

closeDoor(e)

- **Precondition**: the door of the car `e` is open.
- **Effect**: the door closes.

Network

deliver(m)

- **Effect**: performs the reaction that matches the kind of message `m`: `receiveHallCall`, `receiveAssign`, `receiveStatus` or `receiveServed`.

deliverAny

- **Precondition**: the network is not empty.
- **Effect**: one message from the network is chosen non deterministically, this choice is named `m`, and `deliver(m)` is performed.

step

First an elevator named `e`, a floor named `f` and a hall call named `c` are chosen non deterministically. Then exactly one of these happens: `pressHallButton(c)`, `pressCarButton(e, f)`, `assignCall(c)`, `departure(e)`, `travel(e)`, `openDoor(e)`, `closeDoor(e)` or `deliverAny`.

Every non deterministic choice has its own unique name (`e`, `f`, `c`, `target`, `m`), because recorded executions identify choices by name.

Safety Invariants

- **INV-01 doorsClosedWhileRunning**: a moving car always has its door closed.
- **INV-02 staysInShaft**: every car is always on a floor that exists.
- **INV-03 runningHasHeading**: a moving car is never idle.
- **INV-04 runningHasTargetAhead**: a moving car always has some target ahead of it in the direction it is going.
- **INV-05 noDoubleService**: no hall call belongs to more than one elevator, counting both the elevators' hall stops and assignment messages on the way.
- **INV-06 assignmentConsistent**: when a call is among an elevator's hall stops, the dispatcher has it assigned to that same elevator.
- **INV-07 noLostHallCall**: every lit lamp is tracked in exactly one place: as a hall call message on the way, as pending, or as assigned.
- **INV-08 assignedCallIsTracked**: every assigned call is on its way to its elevator, held in that elevator's hall stops, or on its way back as a served message.
- **INV-09 noTrackingWithoutLamp**: the dispatcher never tracks a call, pending or assigned, whose lamp is off.
- **safety**: all of the above together.

Witnesses

Each witness is written as an invariant stating that the situation never happens, so it is expected to be violated: a violation proves that the situation can happen.

- **WIT-01 witnessBothRunning**: both cars are moving at the same time.
- **WIT-02 witnessHallCallServed**: some served message is in the network.
- **WIT-03 witnessStaleKnowledge**: the dispatcher believes some elevator is on a different floor than it really is.

Test Scenarios

`call2Down` is the hall call on floor 2 going down. Every test starts from the initial state. Messages are delivered with `deliver`, giving the exact message. "Not possible" means the test ends with the expectation that the last step fails.

- **hallCallLifecycleTest**: the hall button for `call2Down` is pressed and its message delivered, so it is the only pending call. It is assigned to "A" with `assignTo` and the assignment delivered. "A" departs going up, travels (still running), travels again (now on floor 2 and stopped), and opens its doors (door open, heading idle). The served message is delivered. At the end no lamp is lit, nothing is assigned, and `safety` holds.
- **litButtonIsIgnoredTest**: pressing the hall button for `call2Down` twice in a row is not possible.
- **noDepartureWithOpenDoorTest**: in "A" floor 3 is selected; "A" departs and travels once; floor 0 is selected; "A" travels twice (now on floor 3) and opens its doors. Departing now is not possible.
- **scanKeepsDirectionTest**: in "A" floor 2 is selected; "A" departs and travels once; floor 0 is selected; "A" travels, opens and closes its doors, and departs. It is now going down, and `safety` holds.
- **closestElevatorIsChosenTest**: in "B" floor 3 is selected; "B" departs and travels once; the dispatcher receives the report that "B" is on floor 1 (report number 1); the hall button for floor 2 going up is pressed and its message delivered. Assigning that call to "A" with `assignTo` is not possible.

Bounds for Model Checking

- 4 floors and 2 elevators.
- Random simulation of up to 60 steps per execution must never violate `safety`.

- Random simulation must find a violation of every witness.

Output

A single file `spec/elevator.qnt` with one module named `elevator`, comments in English, using exactly the names given above. Nothing else.

`docs/DESIGN_IMPL.md` is used in Phase 4. It is pasted together with the verified spec and describes exactly what C++ API to generate and what every method must do.

docs/DESIGN_IMPL.md

```
Here is a verified Quint spec for an elevator group: [paste spec/elevator.qnt]
Here is the original design doc: [paste docs/DESIGN_SPEC.md]

Generate a C++20 implementation with these requirements:

1. Types in `include/elevator/types.hpp`, namespace `elevator`, mirroring the Quint types:
   - `using ElevatorId = std::string` for elevator names
   - `enum class Dir { Up, Down }`, `Door { Open, Closed }`, `Motor { Stopped, Running }`
   - `enum class Heading { GoingUp, GoingDown, Idle }` for `Going(Up) | Going(Down) | Idle`
   - structs `HallCall`, `ElevatorState`, `PositionReport`, `DispatcherState`
     with the same field names as the Quint records
   - message structs with these fields:
     `HallCallMsg { HallCall call; }`, `AssignMsg { ElevatorId elevator; HallCall call; }`,
     `StatusMsg { ElevatorId elevator; int floor; int seq; }`,
     `ServedMsg { ElevatorId elevator; HallCall call; }`,
     and `using Message = std::variant<HallCallMsg, AssignMsg, StatusMsg, ServedMsg>`
   - `struct SystemState` with one field per Quint state variable, same names:
     `std::map<ElevatorId, ElevatorState> elevators`, `DispatcherState dispatcher`,
     `std::set<HallCall> hallLamps`, `std::set<Message> network`
   - Quint `Set` -> `std::set`, Quint map -> `std::map`
   - defaulted `operator<=>` on `HallCall` and the message structs (they are
     stored in `std::set`), defaulted `operator==` on all other structs
   - `struct Config { int numFloors = 4; std::vector<ElevatorId> elevators = {"A", "B"};
     int bottom() const; int top() const; }`, where `bottom()` is 0 and
     `top()` is `numFloors - 1`

2. A class `ElevatorSystem` in `include/elevator/elevator_system.hpp`:
   - `explicit ElevatorSystem(Config cfg = {})`, which calls `init()`
   - `const Config& config() const` and `const SystemState& state() const`

3. Methods corresponding to each Quint action, same name, same parameters.
   Every method returns false and leaves the state untouched if the
   precondition fails; otherwise it applies the effect and returns true.
   A method given an unknown elevator name returns false. Anything a method
   does not mention stays unchanged.

   - `void init()`: puts the system into the initial state. Every car is on
     the bottom floor with the door closed, the motor stopped, heading `Idle`,
     no car calls, no hall stops and `statusSeq` 0. The dispatcher has nothing
     pending, nothing assigned, and knows every elevator to be on the bottom
     floor with `seq` 0. No hall lamp is lit and the network is empty. The
     constructor calls it.

   - `bool pressHallButton(HallCall c)`: a passenger presses a hall button.
     Precondition: `c` is a valid hall call and its lamp is not lit.
     Effect: the lamp of `c` is lit and a `HallCallMsg` with `c` is put into
     the network.

   - `bool pressCarButton(ElevatorId e, int f)`: a passenger in car `e`
     selects floor `f`; allowed while the car is moving.
     Precondition: `f` is a floor of the building, it is not the car's
     current floor, and it is not already among the car's `carCalls`.
     Effect: `f` is added to the car's `carCalls`.
```

```
- `bool receiveHallCall(HallCall c)`: the dispatcher receives a hall call.
  Precondition: a `HallCallMsg` with `c` is in the network.
  Effect: that message is removed from the network. If `c` is neither in
  `pending` nor a key of `assigned`, it is added to `pending`; otherwise
  the dispatcher does not change.

- `bool assignTo(HallCall c, ElevatorId e)`: the dispatcher gives a pending
  call to elevator `e`. This also implements Quint `assignCall(c)`, whose
  nondeterministic choice `target` is passed in as `e`.
  Precondition: `c` is in `pending` and `e` is one of
  `closestElevators(dispatcher, c.floor)`.
  Effect: `c` is removed from `pending`, `assigned[c]` becomes `e`, and an
  `AssignMsg` for `e` and `c` is put into the network.

- `bool receiveStatus(StatusMsg s)`: the dispatcher receives a position
  report.
  Precondition: `s` is in the network.
  Effect: `s` is removed from the network. If `s.seq` is greater than the
  `seq` in `known[s.elevator]`, then `known[s.elevator]` becomes `s.floor`
  and `s.seq`. Otherwise the report is outdated and is ignored.

- `bool receiveServed(ServedMsg s)`: the dispatcher learns that a call was
  served.
  Precondition: `s` is in the network.
  Effect: `s` is removed from the network, `s.call` is removed from
  `assigned`, and the lamp of `s.call` goes off.

- `bool receiveAssign(AssignMsg a)`: an elevator receives an assignment.
  Precondition: `a` is in the network.
  Effect: `a` is removed from the network and `a.call` is added to the
  `hallStops` of the car `a.elevator`.

- `bool departure(ElevatorId e)`: car `e` starts moving.
  Precondition: the motor is stopped, the door is closed, the car has at
  least one target, and its current floor is not one of its targets.
  Effect: the motor is running and the heading becomes `GoingUp` or
  `GoingDown` according to `nextDir` of the car.

- `bool travel(ElevatorId e)`: car `e` moves by one floor.
  Precondition: the motor is running.
  Effect: the new floor is the current floor plus `delta` of the heading.
  The car's floor becomes the new floor. If the new floor is one of the
  car's targets, computed before the move, the motor stops; otherwise it
  keeps running. `statusSeq` increases by one, and a `StatusMsg` with the
  car's name, the new floor and the new `statusSeq` is put into the
  network.

- `bool openDoor(ElevatorId e)`: car `e` opens its doors at a target floor.
  Precondition: the motor is stopped, the door is closed, and the current
  floor is one of the car's targets.
  Effect: the served calls are the car's `hallStops` on the current floor.
  The current floor is removed from `carCalls` and the served calls are
  removed from `hallStops`. The door opens. If the car has no targets left,
  the heading becomes `Idle`; otherwise it stays as it was. For every
  served call a `ServedMsg` with the car's name and that call is put into
  the network.

- `bool closeDoor(ElevatorId e)`: car `e` closes its doors.
  Precondition: the door is open.
  Effect: the door is closed.

- `bool deliver(Message m)`: the network delivers message `m`. This also
  implements Quint `deliverAny`, whose nondeterministic choice `m` is
  passed in.
  Effect: calls `receiveHallCall`, `receiveAssign`, `receiveStatus` or
  `receiveServed` depending on the kind of `m`, and returns its result.

`assignCall`, `deliverAny` and `step` get no method of their own: their
nondeterministic choice is made by the caller.
```

```
4. Public const accessors for every Quint state variable, returning a const
   reference to the matching `SystemState` field:
   - `elevators() const`   (maps to Quint var `elevators`)
   - `dispatcher() const`  (maps to Quint var `dispatcher`)
   - `hallLamps() const`   (maps to Quint var `hallLamps`)
   - `network() const`     (maps to Quint var `network`)
   These accessors MUST be named exactly as the Quint variables are named,
   because the C++ test driver will compare them to ITF trace state by name.

5. Pure helpers as free functions with the Quint names, declared in
   `elevator_system.hpp`:
   - `int delta(Heading)`: 1 for `GoingUp`, -1 for `GoingDown`, 0 for `Idle`.
   - `bool isValidHallCall(const Config&, const HallCall&)`: true when the
     floor exists, the call is not `Up` on the top floor, and not `Down` on
     the bottom floor.
   - `std::vector<HallCall> allHallCalls(const Config&)` (Quint `HALL_CALLS`):
     every valid hall call.
   - `std::set<int> targets(const ElevatorState&)`: the car's `carCalls`
     together with the floors of all its `hallStops`.
   - `Dir nextDir(const ElevatorState&)`, the SCAN rule: if the car is going
     up and has a target above it, `Up`; otherwise, if it is going down and
     has a target below it, `Down`; otherwise, if it has any target above it,
     `Up`; otherwise `Down`.
   - `std::set<ElevatorId> closestElevators(const Config&, const DispatcherState&, int floor)`:
     the elevators whose floor in `known` is at the smallest distance from
     `floor`; all of them when several are equally close. Uses only the
     dispatcher's knowledge.

6. Preconditions enforced identically to the Quint guards. `travel` has no
   shaft-bounds check, exactly like the spec.

7. Comments on each method citing the Quint action name it implements.

8. Implementation split by participant:
   - `src/elevator_system.cpp`: helpers, `init`, `deliver`
   - `src/environment.cpp`: `pressHallButton`, `pressCarButton`
   - `src/dispatcher.cpp`: `receiveHallCall`, `assignTo`, `receiveStatus`, `receiveServed`
   - `src/controller.cpp`: `receiveAssign`, `departure`, `travel`, `openDoor`, `closeDoor`

9. `include/` and `src/` contain production code only: no invariants, no JSON,
   no trace parsing, no test helpers.
```

---

3. Phase 2 — Claude Generates the Quint Spec

What to ask Claude

```
Here is the design doc: [paste docs/DESIGN_SPEC.md]

Generate spec/elevator.qnt exactly as the doc requires. Nothing else.
```

The result is a 558-line spec (full text in the Appendix). The next section walks through it.

---

4. The Quint Spec: Actions and Invariants Explained

Every action is a guard (conditions that must hold) followed by the new values of all four state variables: `elevators`, `dispatcher`, `hallLamps` and `network`. An action whose guard is false simply cannot happen in that state.

Helpers the actions rely on

Every floor where the car must stop: floors pressed inside the car plus floors of the hall calls it was given. Most guards and invariants are phrased in terms of targets.

```
// All floors at which the car has to stop.
pure def targets(es: ElevatorState): Set[int] =
  es.carCalls.union(es.hallStops.map(c => c.floor))
```

The SCAN rule. A car keeps its direction while there is work ahead and only then turns around. This is what prevents a car from oscillating between two passengers and starving the others.

```
// SCAN: keep going in the current direction while there is work ahead,
// otherwise reverse. Only meaningful when the car has a target elsewhere.
pure def nextDir(es: ElevatorState): Dir = {
  val ts = targets(es)
  val above = ts.exists(t => t > es.floor)
  val below = ts.exists(t => t < es.floor)
  if (es.heading == Going(Up) and above) Up
  else if (es.heading == Going(Down) and below) Down
  else if (above) Up
  else Down
}
```

The dispatcher's choice. It is computed from `dispatcher.known`, the last position reported by each car, never from the real position. That models the real situation where the dispatcher only knows what the network told it, possibly too late.

```
// Elevators with the minimal distance (by last known position) to a floor.
pure def closestElevators(d: DispatcherState, floor: int): Set[str] =
  ELEVATORS.filter(e =>
    ELEVATORS.forall(o =>
      abs(d.known.get(e).floor - floor) <= abs(d.known.get(o).floor - floor)
    )
  )
```

Actions

INITIAL STATE

`init`: Both cars stand on the ground floor with closed doors and no work. The dispatcher believes exactly that, with report number 0, so its knowledge starts correct and can only become stale later.

```
action init = all {
  elevators' = ELEVATORS.mapBy(_ => {
    floor: BOTTOM,
    door: Closed,
    motor: Stopped,
    heading: Idle,
    carCalls: Set(),
    hallStops: Set(),
    statusSeq: 0,
  }),
  dispatcher' = {
    pending: Set(),
    assigned: Map(),
    known: ELEVATORS.mapBy(_ => { floor: BOTTOM, seq: 0 }),
  },
  hallLamps' = Set(),
  network' = Set(),
}
```

PASSENGERS

`pressHallButton`: A passenger presses a hall button. The guard forbids pressing a lit button, which models a real panel: the second press does nothing. The panel does not talk to any car directly; it only puts a `HallCallMsg` into the network.

```
// A passenger presses a hall button. Pressing an already lit button has no effect.
action pressHallButton(c: HallCall): bool = all {
  HALL_CALLS.contains(c),
  not(hallLamps.contains(c)),
  hallLamps' = hallLamps.union(Set(c)),
  network' = network.union(Set(HallCallMsg(c))),
  elevators' = elevators,
  dispatcher' = dispatcher,
}
```

`pressCarButton`: A passenger inside a car selects a floor. It is handled locally by the car and is allowed while the car is moving, which is one of the interleavings that makes the model interesting: new work can appear behind or ahead of a running car.

```
// A passenger inside car `e` presses the button for floor `f`.
action pressCarButton(e: str, f: int): bool = {
  val es = elevators.get(e)
  all {
    FLOORS.contains(f),
    f != es.floor,
    not(es.carCalls.contains(f)),
    elevators' = elevators.set(e, { ...es, carCalls: es.carCalls.union(Set(f)) }),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
    network' = network,
  }
}
```

DISPATCHER

`receiveHallCall`: The dispatcher takes a hall call message out of the network. If the call is already pending or assigned, nothing happens, so the dispatcher never tracks the same call twice.

```
action receiveHallCall(c: HallCall): bool = {
  val known = dispatcher.pending.contains(c) or dispatcher.assigned.keys().contains(c)
  all {
    network.contains(HallCallMsg(c)),
    network' = network.exclude(Set(HallCallMsg(c))),
    dispatcher' = if (known) dispatcher
                  else { ...dispatcher, pending: dispatcher.pending.union(Set(c)) },
    elevators' = elevators,
    hallLamps' = hallLamps,
  }
}
```

`assignTo`: The dispatcher gives a pending call to one of the closest elevators and sends it an `AssignMsg`. The guard makes the closest-elevator rule part of the protocol: assigning to a farther car is simply not possible.

```
// Assigns a pending call to a specific elevator. Only the closest elevators qualify.
action assignTo(c: HallCall, e: str): bool = all {
  dispatcher.pending.contains(c),
  closestElevators(dispatcher, c.floor).contains(e),
  dispatcher' = {
    ...dispatcher,
    pending: dispatcher.pending.exclude(Set(c)),
    assigned: dispatcher.assigned.put(c, e),
  },
  network' = network.union(Set(AssignMsg({ elevator: e, call: c }))),
  elevators' = elevators,
  hallLamps' = hallLamps,
}
```

`assignCall`: When two cars are equally close, the choice is nondeterministic and named `target`. The trace records it, so the C++ driver calls the deterministic `assignTo(c, target)`.

```
// Ties between equally close elevators are resolved non-deterministically.
action assignCall(c: HallCall): bool = {
  nondet target = closestElevators(dispatcher, c.floor).oneOf()
  assignTo(c, target)
}
```

`receiveStatus`: A position report arrives. Because the network may reorder messages, an older report can arrive after a newer one. The sequence number check discards it; without it the dispatcher would move a car backwards in its mind. This is the guard the injected bug in Phase 7 removes.

```
action receiveStatus(s: { elevator: str, floor: int, seq: int }): bool = {
  val fresh = s.seq > dispatcher.known.get(s.elevator).seq
  all {
    network.contains(StatusMsg(s)),
    network' = network.exclude(Set(StatusMsg(s))),
    dispatcher' = if (fresh)
                    { ...dispatcher,
                      known: dispatcher.known.set(s.elevator, { floor: s.floor, seq: s.seq }) }
                  else dispatcher,
    elevators' = elevators,
    hallLamps' = hallLamps,
  }
}
```

`receiveServed`: The car confirms a served call. Only now does the lamp go off and the assignment disappear, so the lamp always means: this call is still somewhere in the system.

```
// The call is confirmed as served: forget it and switch the hall lamp off.
action receiveServed(s: { elevator: str, call: HallCall }): bool = all {
  network.contains(ServedMsg(s)),
  network' = network.exclude(Set(ServedMsg(s))),
  dispatcher' = { ...dispatcher, assigned: without(dispatcher.assigned, s.call) },
  hallLamps' = hallLamps.exclude(Set(s.call)),
  elevators' = elevators,
}
```

ELEVATOR CONTROLLER

`receiveAssign`: A car receives an assignment and adds the call to its hall stops. Between `assignTo` and this step the call exists only as a message in flight.

```
action receiveAssign(a: { elevator: str, call: HallCall }): bool = {
  val es = elevators.get(a.elevator)
  all {
    network.contains(AssignMsg(a)),
    network' = network.exclude(Set(AssignMsg(a))),
    elevators' = elevators.set(a.elevator,
                               { ...es, hallStops: es.hallStops.union(Set(a.call)) }),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
  }
}
```

`departure`: A stopped car with closed doors and work elsewhere starts moving in the direction chosen by SCAN. Work on the current floor must be done first by opening the doors, so a car never leaves a floor it was supposed to serve.

```
// The car starts moving towards its next target. Work on the current floor
// must be done first (see openDoor).
action departure(e: str): bool = {
  val es = elevators.get(e)
  val ts = targets(es)
  all {
    es.motor == Stopped,
    es.door == Closed,
    ts != Set(),
    not(ts.contains(es.floor)),
    elevators' = elevators.set(e, { ...es, motor: Running, heading: Going(nextDir(es)) }),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
    network' = network,
  }
}
```

`travel`: The car moves one floor, stops if that floor is a target, increments its report number and reports its position. There is deliberately no check that the new floor exists: staying inside the building must follow from the other rules, and `staysInShaft` proves it.

```
// The car moves by one floor. If the new floor is a target, it stops there.
// Every floor change is reported to the dispatcher.
action travel(e: str): bool = {
  val es = elevators.get(e)
  val newFloor = es.floor + delta(es.heading)
  val seq = es.statusSeq + 1
  all {
    es.motor == Running,
    elevators' = elevators.set(e, {
      ...es,
      floor: newFloor,
      motor: if (targets(es).contains(newFloor)) Stopped else Running,
      statusSeq: seq,
    }),
    network' = network.union(Set(StatusMsg({ elevator: e, floor: newFloor, seq: seq }))),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
  }
}
```

`openDoor`: At a target floor the car opens its doors and serves everything there: the car call and all hall calls for that floor, each confirmed with a `ServedMsg`. The heading is reset to `Idle` only when no work remains, otherwise SCAN keeps its direction.

```
// The car opens its doors at a target floor. All car calls and hall calls
// for this floor are served at once.
action openDoor(e: str): bool = {
  val es = elevators.get(e)
  val served = es.hallStops.filter(c => c.floor == es.floor)
  val remaining = { ...es,
                    carCalls: es.carCalls.exclude(Set(es.floor)),
                    hallStops: es.hallStops.exclude(served) }
  all {
    es.motor == Stopped,
    es.door == Closed,
    targets(es).contains(es.floor),
    elevators' = elevators.set(e, {
      ...remaining,
      door: Open,
      heading: if (targets(remaining) == Set()) Idle else es.heading,
    }),
    network' = network.union(served.map(c => ServedMsg({ elevator: e, call: c }))),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
  }
}
```

`closeDoor`: Closing the doors is always possible when they are open. Together with `departure` requiring closed doors, this gives `doorsClosedWhileRunning`.

```
action closeDoor(e: str): bool = {
  val es = elevators.get(e)
  all {
    es.door == Open,
    elevators' = elevators.set(e, { ...es, door: Closed }),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
    network' = network,
  }
}
```

NETWORK

`deliver`: Delivery dispatches a message to the reaction for its kind.

```
action deliver(m: Message): bool =
  match m {
    | HallCallMsg(c) => receiveHallCall(c)
    | AssignMsg(a) => receiveAssign(a)
    | StatusMsg(s) => receiveStatus(s)
    | ServedMsg(s) => receiveServed(s)
  }
```

`deliverAny`: Any message in flight may be delivered next, which is how the model expresses an asynchronous network with arbitrary delays and reordering. The chosen message is recorded as the nondeterministic pick `m`.

```
action deliverAny: bool = all {
  network != Set(),
  nondet m = network.oneOf()
  deliver(m),
}
```

TRANSITION RELATION

`step`: Every step first picks an elevator `e`, a floor `f` and a hall call `c`, then exactly one enabled action happens. The names of these picks, and of `target` and `m`, are what `mbt::nondetPicks` contains in the trace.

```
action step = {
  nondet e = ELEVATORS.oneOf()
  nondet f = FLOORS.oneOf()
  nondet c = HALL_CALLS.oneOf()
  any {
    pressHallButton(c),
    pressCarButton(e, f),
    assignCall(c),
    departure(e),
    travel(e),
    openDoor(e),
    closeDoor(e),
    deliverAny,
  }
}
```

Safety invariants: what they say and why they are there

`doorsClosedWhileRunning`: Passenger safety: a car never moves with open doors. It follows from `departure` requiring closed doors and `openDoor` requiring a stopped car. A bug in either guard breaks it immediately.

```
// A car never moves with open doors.
val doorsClosedWhileRunning =
  ELEVATORS.forall(e => elevators.get(e).motor == Running implies elevators.get(e).door == Closed)
```

`staysInShaft`: A car never goes below the ground floor or above the top floor. `travel` has no bounds check, so this is a real property of the algorithm, not of a defensive if-statement. It holds because a moving car always has a target ahead of it (the next invariant).

```
// A car never leaves the shaft.
val staysInShaft =
  ELEVATORS.forall(e => FLOORS.contains(elevators.get(e).floor))
```

`runningHasHeading`: A moving car always has a direction. With heading `Idle`, `delta` is 0 and the car would run forever without moving. It holds because only `departure` starts the motor and it always sets a direction.

```
// A running car always has a direction.
val runningHasHeading =
  ELEVATORS.forall(e => elevators.get(e).motor == Running implies elevators.get(e).heading != Idle)
```

`runningHasTargetAhead`: A moving car always has some target strictly ahead of it. This is the key fact behind `staysInShaft` and behind cars not running aimlessly. Targets are removed only by `openDoor`, which requires a stopped car, so a target ahead cannot disappear while moving.

```
// A running car always has a target ahead of it, so it never runs aimlessly.
val runningHasTargetAhead =
  ELEVATORS.forall(e => {
    val es = elevators.get(e)
    es.motor == Running implies targets(es).exists(t => (t - es.floor) * delta(es.heading) > 0)
  })
```

`noDoubleService`: No hall call belongs to two elevators at once. Ownership is counted both in the cars' hall stops and in assignment messages still in flight. It checks that the dispatcher assigns every call exactly once even though messages are delayed.

```
// Every hall call is owned by at most one elevator (in flight or accepted).
val noDoubleService =
  HALL_CALLS.forall(c => {
    val owners = ELEVATORS.filter(e =>
      elevators.get(e).hallStops.contains(c)
        or network.contains(AssignMsg({ elevator: e, call: c }))
    )
    owners.size() <= 1
  })
```

`assignmentConsistent`: Every hall stop of a car matches the dispatcher's assignment. A car never serves a call the dispatcher gave to someone else or already forgot about.

```
// The dispatcher's assignment always matches the elevator that owns the call.
val assignmentConsistent =
  ELEVATORS.forall(e =>
    elevators.get(e).hallStops.forall(c =>
      dispatcher.assigned.keys().contains(c) and dispatcher.assigned.get(c) == e
    )
  )
```

`noLostHallCall`: Every lit lamp is tracked in exactly one place: as a message to the dispatcher, as pending, or as assigned. "At least one" means no call is ever lost in the asynchronous pipeline; "at most one" means no call is duplicated, for example by a repeated hall call message.

```
// No lost calls: every lit hall lamp is tracked by exactly one stage of the
// pipeline (message to the dispatcher, pending, or assigned).
val noLostHallCall =
  hallLamps.forall(c => {
    val inFlight = if (network.contains(HallCallMsg(c))) 1 else 0
    val pending = if (dispatcher.pending.contains(c)) 1 else 0
    val assigned = if (dispatcher.assigned.keys().contains(c)) 1 else 0
    inFlight + pending + assigned == 1
  })
```

`assignedCallIsTracked`: Every assigned call is on its way to its car, held by it, or on its way back as a served message. An assignment can never get stuck: there is always a message or a car responsible for it.

```
// Every assigned call is on its way to, held by, or confirmed by its elevator.
val assignedCallIsTracked =
  dispatcher.assigned.keys().forall(c => {
    val e = dispatcher.assigned.get(c)
    network.contains(AssignMsg({ elevator: e, call: c }))
      or elevators.get(e).hallStops.contains(c)
      or network.contains(ServedMsg({ elevator: e, call: c }))
  })
```

`noTrackingWithoutLamp`: The dispatcher never tracks a call whose lamp is off. The reverse direction of `noLostHallCall`: the lamp goes off only after the dispatcher has forgotten the call, never before.

```
// A lamp is switched off only after the call has been served.
val noTrackingWithoutLamp =
  dispatcher.pending.union(dispatcher.assigned.keys()).forall(c => hallLamps.contains(c))
```

`safety`: All nine invariants together. This is the single property passed to `quint run --invariant=safety`, both in simulation and during trace generation.

```
val safety = and {
  doorsClosedWhileRunning,
  staysInShaft,
  runningHasHeading,
  runningHasTargetAhead,
  noDoubleService,
  assignmentConsistent,
  noLostHallCall,
  assignedCallIsTracked,
  noTrackingWithoutLamp,
}
```

Witnesses

Witnesses are not safety properties. Each one claims that an *interesting* situation never happens, and is expected to be violated. Phase 3 explains why they exist and how to read their output.

```
// Both cars are moving at the same time.
val witnessBothRunning =
  not(ELEVATORS.forall(e => elevators.get(e).motor == Running))
```

```
// Some elevator has served a hall call and is reporting it to the dispatcher.
val witnessHallCallServed =
  not(network.exists(m => match m { | ServedMsg(_) => true | _ => false }))
```

```
// The dispatcher acts on a stale position of some elevator.
val witnessStaleKnowledge =
  ELEVATORS.forall(e => dispatcher.known.get(e).floor == elevators.get(e).floor)
```

---

5. Phase 3 — Quint Model-Checks the Spec

Before any C++ exists, the spec must be correct on its own. These commands check **only the Quint spec**; no C++ code is involved yet:

```
quint typecheck spec/elevator.qnt
quint test spec/elevator.qnt
quint run spec/elevator.qnt --invariant=safety --max-samples=1000 --max-steps=60
quint run spec/elevator.qnt --invariant=witnessBothRunning --max-steps=60
quint run spec/elevator.qnt --invariant=witnessHallCallServed --max-steps=60
quint run spec/elevator.qnt --invariant=witnessStaleKnowledge --max-steps=60
```

Command	Question it answers	Good result
`quint typecheck`	Is the spec well-formed?	no errors
`quint test`	Do the 5 hand-written `run` scenarios behave as described in the design doc?	`5 passing`
`--invariant=safety`	Can something **bad** ever happen?	`[ok] No violation found`
`--invariant=witness...`	Can something **interesting** happen at all?	`[violation]` with an example

Why witnesses exist

A passing safety check is only meaningful if the model actually reaches the situations where things could go wrong. Imagine a mistake in the spec where `travel` updated the dispatcher's knowledge directly instead of sending a `StatusMsg` through the network. Every safety invariant would still pass, but only because stale knowledge could never occur: the `seq` check in `receiveStatus` would never be exercised, the generated traces would never contain reordered status reports, and the bug injected in Phase 7 would never be found. The checks would be green *for the wrong reason*.

Witnesses guard against that. Quint has no separate "find me an example" command, so a witness is written the other way round: as an invariant claiming that the interesting situation *never* happens. When Quint violates it, the violation is the proof, and the counterexample is a concrete execution that reaches the situation. A witness that is *not* violated is the alarm: the model is too weak and the safety results cannot be trusted.

So a red `[violation]` for a witness says nothing bad about the system, and nothing at all about the C++ code. It is a one-time sanity check of the model itself, done when the spec is written or changed.

The three witnesses

Witness	Situation that must be reachable	What its violation proves	Result (seed 0xd0e6e75026a2eb30)
`witnessBothRunning`	both cars are moving at the same time	the two elevators really run concurrently, so invariants like `noDoubleService` are tested with two cars competing for calls	`[violation] Found an issue` in state 31
`witnessHallCallServed`	a hall call has been served and the confirmation is in flight	the whole pipeline (button, dispatcher, assignment, travel, door, confirmation) can complete, so the lamp and assignment invariants are checked on calls that actually finish	`[violation] Found an issue` in state 60
`witnessStaleKnowledge`	the dispatcher believes some elevator is on a different floor than it really is	the asynchronous network really delays status reports, so the `seq` check in `receiveStatus` and the closest-elevator choice on outdated data are exercised	`[violation] Found an issue` in state 7

Reading a witness result

```
// The dispatcher acts on a stale position of some elevator.
val witnessStaleKnowledge =
  ELEVATORS.forall(e => dispatcher.known.get(e).floor == elevators.get(e).floor)
```

The witness claims "the dispatcher always knows the real floor of every elevator". Quint violates it in state 7:

```
$ quint run spec/elevator.qnt --invariant=witnessStaleKnowledge --max-steps=60 --
seed=0xd0e6e75026a2eb30
```

```
[State 7]
{
  dispatcher:
    {
      assigned: Map(),
      known: Map("A" -> { floor: 0, seq: 0 }, "B" -> { floor: 0, seq: 0 }),
      pending: Set({ dir: Up, floor: 2 })
    },
  elevators:
    Map(
      "A" ->
        {
```

```
          carCalls: Set(2),
          door: Closed,
          floor: 0,
          hallStops: Set(),
          heading: Idle,
          motor: Stopped,
          statusSeq: 0
        },
      "B" ->
        {
          carCalls: Set(2),
          door: Closed,
          floor: 1,
          hallStops: Set(),
          heading: Going(Up),
          motor: Running,
          statusSeq: 1
        }
    ),
  hallLamps: Set({ dir: Up, floor: 1 }, { dir: Up, floor: 2 }),
  network:
    Set(
      HallCallMsg({ dir: Up, floor: 1 }),
      StatusMsg({ elevator: "B", floor: 1, seq: 1 })
    )
}

[violation] Found an issue (17ms at 59 traces/second).
```

- Elevator **B** really is on **floor 1** and running up (`floor: 1, motor: Running`).
- The dispatcher still believes B is on **floor 0** (`known: "B" -> { floor: 0, seq: 0 }`).
- B's report `StatusMsg({ elevator: "B", floor: 1, seq: 1 })` is still in the `network`, not delivered yet.
- Meanwhile a hall call is already `pending`, so the dispatcher is about to assign it using outdated positions.

This is exactly the behaviour of a real system with an asynchronous network, and exactly the situation in which the Phase 7 bug shows up. The model reaches it in just a few steps, so the safety checks and the generated traces cover it.

A modelling error found in this phase

This phase already caught a modelling error: an early invariant `idleMeansNoWork` ("an idle car has no targets") was violated, because a stopped car can receive a new assignment before it departs. The invariant was wrong, not the model; the `Heading` comment now states this explicitly.

---

6. Phase 4 — Claude Generates the C++ Implementation

include/elevator/elevator_system.hpp

```
#pragma once

#include <map>
#include <set>
#include <string>
#include <vector>

#include "elevator/types.hpp"
```

```hpp
namespace elevator {

// Pure helpers (Quint `pure def`).
int delta(Heading h);
bool isValidHallCall(const Config& cfg, const HallCall& c);
std::vector<HallCall> allHallCalls(const Config& cfg);
std::set<int> targets(const ElevatorState& es);
Dir nextDir(const ElevatorState& es);
std::set<ElevatorId> closestElevators(const Config& cfg, const DispatcherState& d, int floor);

// The elevator group control system. Every public action mirrors the Quint
// action of the same name: if its guard does not hold it returns false and
// leaves the state untouched, otherwise it applies the effect and returns true.
class ElevatorSystem {
public:
    explicit ElevatorSystem(Config cfg = {});

    const Config& config() const { return cfg_; }
    const SystemState& state() const { return s_; }

    // One accessor per Quint state variable, named exactly like the variable.
    const std::map<ElevatorId, ElevatorState>& elevators() const { return s_.elevators; }
    const DispatcherState& dispatcher() const { return s_.dispatcher; }
    const std::set<HallCall>& hallLamps() const { return s_.hallLamps; }
    const std::set<Message>& network() const { return s_.network; }

    // Quint: init
    void init();

    // Quint: pressHallButton(c)
    bool pressHallButton(const HallCall& c);
    // Quint: pressCarButton(e, f)
    bool pressCarButton(const ElevatorId& e, int f);

    // Quint: receiveHallCall(c)
    bool receiveHallCall(const HallCall& c);
    // Quint: assignTo(c, e); assignCall(c) picks e and calls this
    bool assignTo(const HallCall& c, const ElevatorId& e);
    // Quint: receiveStatus(s)
    bool receiveStatus(const StatusMsg& s);
    // Quint: receiveServed(s)
    bool receiveServed(const ServedMsg& s);

    // Quint: receiveAssign(a)
    bool receiveAssign(const AssignMsg& a);
    // Quint: departure(e)
    bool departure(const ElevatorId& e);
    // Quint: travel(e)
    bool travel(const ElevatorId& e);
    // Quint: openDoor(e)
    bool openDoor(const ElevatorId& e);
    // Quint: closeDoor(e)
    bool closeDoor(const ElevatorId& e);

    // Quint: deliver(m); deliverAny picks m and calls this
    bool deliver(const Message& m);

private:
    bool hasElevator(const ElevatorId& e) const;

    Config cfg_;
    SystemState s_;
};

}  // namespace elevator
```

Each method is a direct translation of the Quint guard and effect:

src/controller.cpp, src/dispatcher.cpp (excerpt)

```
// No shaft-bounds guard on purpose: a running car always has a target ahead,
// so it never moves past the last floor.
bool ElevatorSystem::travel(const ElevatorId& e) {
    if (!hasElevator(e)) return false;
    auto& es = s_.elevators.at(e);
    if (es.motor != Motor::Running) return false;
    const int newFloor = es.floor + delta(es.heading);
    const bool arrive = targets(es).contains(newFloor);
    es.floor = newFloor;
    es.motor = arrive ? Motor::Stopped : Motor::Running;
    es.statusSeq += 1;
    s_.network.insert(StatusMsg{e, newFloor, es.statusSeq});
    return true;
}

bool ElevatorSystem::receiveStatus(const StatusMsg& s) {
    if (!s_.network.contains(s)) return false;
    s_.network.erase(s);
    auto& report = s_.dispatcher.known.at(s.elevator);
    if (s.seq > report.seq) report = PositionReport{.floor = s.floor, .seq = s.seq};
    return true;
}
```

---

7. Phase 5 — Quint Emits ITF Traces

```
quint run spec/elevator.qnt --mbt --invariant=safety \
  --max-samples=50 --n-traces=50 --max-steps=60 \
  --out-itf=traces/trace_{seq}.itf.json
```

Yes, traces are generated **with the invariants on**: `--invariant=safety` makes Quint evaluate all nine invariants in every state of every trace it writes, and the command fails if any of them is violated. So every committed trace is a verified correct execution of the spec. `--mbt` adds two extra variables to every state: `mbt::actionTaken` and `mbt::nondetPicks`. Each of the 50 traces has 61 states (init + 60 steps). The invariants are checked once more on the C++ side: after every replayed step the driver evaluates the same nine invariants on the C++ state (`tests/TestSupport.hpp`).

One trace, step by step: trace_42.itf.json

This is state 12 of `traces/trace_42.itf.json`, exactly as Quint wrote it (only re-indented):

traces/trace_42.itf.json: state 12


```
{
  "mbt::actionTaken": "assignCall",
  "mbt::nondetPicks": {
    "c": {
      "tag": "Some",
      "value": {"dir": {"tag": "Up", "value": {"#tup": []}}, "floor": {"#bigint": "2"}}
    },
    "e": {"tag": "Some", "value": "A"},
    "f": {"tag": "Some", "value": {"#bigint": "3"}},
    "m": {"tag": "None", "value": {"#tup": []}},
    "target": {"tag": "Some", "value": "A"}
  },
  "elevators": {
    "#map": [
      [
```

```
        "A",
        {
          "carCalls": {"#set": [{"#bigint": "2"}, {"#bigint": "3"}]},
          "door": {"tag": "Closed", "value": {"#tup": []}},
          "floor": {"#bigint": "1"},
          "hallStops": {"#set": []},
          "heading": {"tag": "Going", "value": {"tag": "Up", "value": {"#tup": []}}},
          "motor": {"tag": "Running", "value": {"#tup": []}},
          "statusSeq": {"#bigint": "1"}
        }
      ],
      [
        "B",
        {
          "carCalls": {"#set": []},
          "door": {"tag": "Closed", "value": {"#tup": []}},
          "floor": {"#bigint": "0"},
          "hallStops": {"#set": []},
          "heading": {"tag": "Idle", "value": {"#tup": []}},
          "motor": {"tag": "Stopped", "value": {"#tup": []}},
          "statusSeq": {"#bigint": "0"}
        }
      ]
    ]
  },
  "dispatcher": {
    "assigned": {
      "#map": [[{"dir": {"tag": "Up", "value": {"#tup": []}}, "floor": {"#bigint": "2"}}, "A"]]
    },
    "known": {
      "#map": [
        ["A", {"floor": {"#bigint": "1"}, "seq": {"#bigint": "1"}}],
        ["B", {"floor": {"#bigint": "0"}, "seq": {"#bigint": "0"}}]
      ]
    },
    "pending": {
      "#set": [
        {"dir": {"tag": "Down", "value": {"#tup": []}}, "floor": {"#bigint": "3"}},
        {"dir": {"tag": "Up", "value": {"#tup": []}}, "floor": {"#bigint": "1"}}
      ]
    }
  },
  "hallLamps": {
    "#set": [
      {"dir": {"tag": "Down", "value": {"#tup": []}}, "floor": {"#bigint": "3"}},
      {"dir": {"tag": "Up", "value": {"#tup": []}}, "floor": {"#bigint": "1"}},
      {"dir": {"tag": "Up", "value": {"#tup": []}}, "floor": {"#bigint": "2"}}
    ]
  },
  "network": {
    "#set": [
      {
        "tag": "AssignMsg",
        "value": {
          "call": {"dir": {"tag": "Up", "value": {"#tup": []}}, "floor": {"#bigint": "2"}},
          "elevator": "A"
        }
      }
    ]
  }
}
```


The same four state variables decoded into Quint syntax, which is what the driver turns into C++ values:

state 12 in Quint syntax

```
elevators  = Map(
  "A" -> { carCalls: Set(2, 3), door: Closed, floor: 1, hallStops: Set(), heading: Going(Up), motor:
Running, statusSeq: 1 },
  "B" -> { carCalls: Set(), door: Closed, floor: 0, hallStops: Set(), heading: Idle, motor: Stopped,
statusSeq: 0 },
)
dispatcher = { assigned: Map({ dir: Up, floor: 2 } -> "A"), known: Map("A" -> { floor: 1, seq: 1 }, "B"
-> { floor: 0, seq: 0 }), pending: Set({ dir: Down, floor: 3 }, { dir: Up, floor: 1 }) }
hallLamps  = Set({ dir: Down, floor: 3 }, { dir: Up, floor: 1 }, { dir: Up, floor: 2 })
network    = Set(AssignMsg({ call: { dir: Up, floor: 2 }, elevator: "A" }))
```

How the driver reads it:

- `"mbt::actionTaken": "assignCall"` tells which branch of `step` fired. The driver maps it to a C++ method (table in Phase 6).
- `"mbt::nondetPicks"` holds every nondeterministic choice of this step as an option. `Some` values are arguments for the call; `None` means the pick was not made in this step. For `assignCall` the driver uses `c` and `target` and calls `assignTo(c, target)`. Picks `e` and `f` are also `Some` because `step` always picks them, but this action does not use them.
- The four state variables are the **expected** state after the step. The driver parses them into C++ types: `{"#bigint": "2"}` becomes `int`, `{"#set": [...]}` becomes `std::set`, `{"#map": [[k, v], ...]}` becomes `std::map`, `{"tag": "Going", "value": {"tag": "Up", ...}}` becomes `Heading::GoingUp`, and a `{"tag": "AssignMsg", ...}` in `network` becomes the `AssignMsg` alternative of the `Message` variant.
- After calling the C++ method, the driver compares `sys.elevators()`, `sys.dispatcher()`, `sys.hallLamps()` and `sys.network()` with these four values, one variable at a time, and then evaluates the invariants on the C++ state.

The first 12 steps of the same trace, as the driver replays them. The last column is the difference between consecutive states in the trace, written in Quint syntax: the C++ call must produce exactly this change and nothing else.

#	mbt::actionTaken	picks used	C++ call made by the driver	state change the driver must reproduce
1	pressHallButton	`c = { dir: Down, floor: 3 }`	`pressHallButton({ dir: Down, floor: 3 })`	`hallLamps + { dir: Down, floor: 3 }` `network + HallCallMsg({ dir: Down, floor: 3 })`
2	deliverAny	`m = HallCallMsg({ dir: Down, floor: 3 })`	`deliver(HallCallMsg({ dir: Down, floor: 3 }))`	`dispatcher.pending + { dir: Down, floor: 3 }` `network − HallCallMsg({ dir: Down, floor: 3 })`
3	pressCarButton	`e = "A", f = 3`	`pressCarButton("A", 3)`	`elevators["A"].carCalls + 3`
4	pressHallButton	`c = { dir: Up, floor: 1 }`	`pressHallButton({ dir: Up, floor: 1 })`	`hallLamps + { dir: Up, floor: 1 }` `network + HallCallMsg({ dir: Up, floor: 1 })`
5	deliverAny	`m = HallCallMsg({ dir: Up, floor: 1 })`	`deliver(HallCallMsg({ dir: Up, floor: 1 }))`	`dispatcher.pending + { dir: Up, floor: 1 }` `network − HallCallMsg({ dir: Up, floor: 1 })`
6	pressCarButton	`e = "A", f = 2`	`pressCarButton("A", 2)`	`elevators["A"].carCalls + 2`

7	pressHallButton	c = { dir: Up, floor: 2 }	pressHallButton({ dir: Up, floor: 2 })	hallLamps + { dir: Up, floor: 2 } network + HallCallMsg({ dir: Up, floor: 2 })
8	deliverAny	m = HallCallMsg({ dir: Up, floor: 2 })	deliver(HallCallMsg({ dir: Up, floor: 2 }))	dispatcher.pending + { dir: Up, floor: 2 } network − HallCallMsg({ dir: Up, floor: 2 })
9	departure	e = "A"	departure("A")	elevators["A"].heading: Idle -> Going(Up) elevators["A"].motor: Stopped -> Running
10	travel	e = "A"	travel("A")	elevators["A"].floor: 0 -> 1 elevators["A"].statusSeq: 0 -> 1 network + StatusMsg({ elevator: "A", floor: 1, seq: 1 })
11	deliverAny	m = StatusMsg({ elevator: "A", floor: 1, seq: 1 })	deliver(StatusMsg({ elevator: "A", floor: 1, seq: 1 }))	dispatcher.known["A"].floor: 0 -> 1 dispatcher.known["A"].seq: 0 -> 1 network − StatusMsg({ elevator: "A", floor: 1, seq: 1 })
12	assignCall	c = { dir: Up, floor: 2 }, target = "A"	assignTo({ dir: Up, floor: 2 }, "A")	dispatcher.assigned[{ dir: Up, floor: 2 }] = "A" dispatcher.pending − { dir: Up, floor: 2 } network + AssignMsg({ call: { dir: Up, floor: 2 }, elevator: "A" })

---

8. Phase 6 — The C++ MBT Driver

`tests/MBTDriver.cpp` is a GoogleTest value-parameterized test: one test case per trace file. For every step it translates the recorded action and picks into a C++ call:

mbt::actionTaken	nondet picks used	C++ call
`pressHallButton`	`c`	`pressHallButton(c)`
`pressCarButton`	`e`, `f`	`pressCarButton(e, f)`
`assignCall`	`c`, `target`	`assignTo(c, target)`
`departure` / `travel` / `openDoor` / `closeDoor`	`e`	same method with `e`
`deliverAny`	`m`	`deliver(m)`

tests/MBTDriver.cpp: applyStep

```
// Translates one ITF transition into the corresponding ElevatorSystem call.
StepResult applyStep(ElevatorSystem& sys, const std::string& action, const json& picks, const
SystemState& expected) {
    auto elevatorPick = [&] { return requirePick(picks, "e").get<std::string>(); };

    if (action == "pressHallButton") {
        const auto c = parseHallCall(requirePick(picks, "c"));
        return {sys.pressHallButton(c), "pressHallButton(" + toJson(c).dump() + ")"};
    }
    if (action == "pressCarButton") {
        const auto e = elevatorPick();
        const int f = toInt(requirePick(picks, "f"));
        return {sys.pressCarButton(e, f), "pressCarButton(" + e + ", " + std::to_string(f) + ")"};
```

```
    }
    if (action == "assignCall") {
        const auto c = parseHallCall(requirePick(picks, "c"));
        std::string target;
        if (const json* t = pick(picks, "target")) {
            target = t->get<std::string>();
        } else if (auto it = expected.dispatcher.assigned.find(c); it !=
expected.dispatcher.assigned.end()) {
            target = it->second;
        } else {
            return {false, "assignCall(" + toJson(c).dump() + "): no target in trace"};
        }
        return {sys.assignTo(c, target), "assignTo(" + toJson(c).dump() + ", " + target + ")"};
    }
    if (action == "departure") {
        const auto e = elevatorPick();
        return {sys.departure(e), "departure(" + e + ")"};
    }
    if (action == "travel") {
        const auto e = elevatorPick();
        return {sys.travel(e), "travel(" + e + ")"};
    }
    if (action == "openDoor") {
        const auto e = elevatorPick();
        return {sys.openDoor(e), "openDoor(" + e + ")"};
    }
    if (action == "closeDoor") {
        const auto e = elevatorPick();
        return {sys.closeDoor(e), "closeDoor(" + e + ")"};
    }
    if (action == "deliverAny") {
        const auto m = parseMessage(requirePick(picks, "m"));
        return {sys.deliver(m), "deliver(" + toJson(m).dump() + ")"};
    }
    throw std::runtime_error("unknown action in trace: " + action);
}
```

After every call the driver checks, in order: the C++ guard accepted the action, every state variable equals the trace (compared through the accessors, by name), and no invariant from `tests/TestSupport.hpp` is violated.

tests/MBTDriver.cpp: comparison and replay loop

```
std::string differingVars(const ElevatorSystem& sys, const SystemState& expected) {
    std::string out;
    auto check = [&](bool same, const char* var) {
        if (!same) out += (out.empty() ? "" : ", ") + std::string(var);
    };
    check(sys.elevators() == expected.elevators, "elevators");
    check(sys.dispatcher() == expected.dispatcher, "dispatcher");
    check(sys.hallLamps() == expected.hallLamps, "hallLamps");
    check(sys.network() == expected.network, "network");
    return out;
}

TEST_P(MBTDriver, ReplaysTrace) {
    const std::string path = GetParam();
    std::ifstream in(path);
    ASSERT_TRUE(in) << "cannot open " << path;
    const json states = json::parse(in).at("states");
    ASSERT_FALSE(states.empty());
    ASSERT_EQ(states.at(0).at("mbt::actionTaken").get<std::string>(), "init")
        << path << ": trace was not generated with --mbt";

    ElevatorSystem sys;
    sys.init();
    const auto initial = parseState(states.at(0));
```

```
    const auto initDiff = differingVars(sys, initial);
    ASSERT_TRUE(initDiff.empty()) << "C++ state differs from Quint state in: " << initDiff
                                  << report(path, 0, "init", "init()", SystemState{}, initial,
sys.state());

    for (size_t i = 1; i < states.size(); ++i) {
        const json& st = states.at(i);
        const auto action = st.at("mbt::actionTaken").get<std::string>();
        const auto expected = parseState(st);
        const auto before = sys.state();

        const auto result = applyStep(sys, action, st.at("mbt::nondetPicks"), expected);
        ASSERT_TRUE(result.enabled) << "C++ rejected an action that Quint took (guard mismatch)"
                                    << report(path, i, action, result.call, before, expected,
sys.state());
        const auto diff = differingVars(sys, expected);
        ASSERT_TRUE(diff.empty()) << "C++ state differs from Quint state in: " << diff
                                  << report(path, i, action, result.call, before, expected,
sys.state());

        const auto violated = violatedInvariants(sys.config(), sys.state());
        ASSERT_TRUE(violated.empty()) << "C++ invariant violated: " << violated.front()
                                      << report(path, i, action, result.call, before, expected,
sys.state());
        ++actionCounts[action];
    }

    ++traces;
    RecordProperty("steps", static_cast<int>(states.size() - 1));
    std::cout << "[   MBT    ] " << fs::path(path).filename().string() << ": " << states.size() - 1
              << " steps replayed" << std::endl;
}
```

---

9. Phase 7 — The Fix Loop: Catching a Real Bug

To prove the loop works we injected a realistic bug into the C++ dispatcher: it stopped discarding outdated position reports.

src/dispatcher.cpp: injected bug

```
- if (s.seq > report.seq) report = PositionReport{.floor = s.floor, .seq = s.seq};
+ report = PositionReport{.floor = s.floor, .seq = s.seq};
```

Of the 11 hand-written unit and scenario tests, only one failed: `StaleStatusIsDiscarded`, which was written specifically for this case. The MBT driver needed no such foresight: it failed **23 of 50 random traces**. The first report:

mbt_driver output (excerpt)

```
C++ state differs from Quint state in: dispatcher
=== MBT DIVERGENCE ===
trace:    traces/trace_0.itf.json
step:     46
action:   deliverAny
C++ call: deliver({"StatusMsg":{"elevator":"B","floor":2,"seq":2}})
--- state before / expected / actual (full JSON, omitted here) ---
--- diff (JSON patch: actual -> expected) ---
[
  {
    "op": "replace",
    "path": "/dispatcher/known/B/floor",
    "value": 3
  },
  {
    "op": "replace",
    "path": "/dispatcher/known/B/seq",
    "value": 3
  }
]

[  FAILED  ] 23 of 50 traces diverged
[   MBT    ] replayed 27 traces, 2374 steps: assignCall=208 closeDoor=153 deliverAny=714 departure=212
openDoor=183 pressCarButton=339 pressHallButton=309 travel=256
```

The report contains everything needed to fix the bug without a human: the action (`deliverAny`, delivering `StatusMsg(B, floor 2, seq 2)`), the variable that diverged (`dispatcher`), and the exact difference: Quint kept the newer report (floor 3, seq 3) while C++ overwrote it with the older one. The two status messages were delivered out of order, an interleaving the simulator found on its own. Given this report and the Quint `receiveStatus` action, Claude restores the `seq` guard and the loop passes again.

What to ask Claude

```
The MBT driver failed. Here is the log: [paste the first === MBT DIVERGENCE === block]
Here is the Quint spec: [paste spec/elevator.qnt]

Find the Quint action named in the report, compare its guard and effect with the
C++ method that was called, and fix the C++ code. Do not modify the spec,
the traces, or the driver.
```

---

10. Running the Whole Loop: Real Output

`scripts/mbt.sh` runs one iteration of the whole loop:

scripts/mbt.sh

```
#!/usr/bin/env bash
# One iteration of the spec -> traces -> C++ replay loop.
#
# Environment:
#   N_TRACES   number of traces to generate        (default 50)
#   MAX_STEPS  steps per trace                     (default 60)
#   SEED       Quint seed, for reproducible traces (default: random)

set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
SPEC="$ROOT/spec/elevator.qnt"
```

```
TRACES="$ROOT/traces"
BUILD="$ROOT/build"
LOG="$BUILD/mbt_last.log"

N_TRACES="${N_TRACES:-50}"
MAX_STEPS="${MAX_STEPS:-60}"
SEED_ARGS=()
if [[ -n "${SEED:-}" ]]; then SEED_ARGS=(--seed="$SEED"); fi

step() { printf '\n==> %s\n' "$*"; }

# Prints only the verdict line on success, the whole counterexample on a violation.
quint_run() {
  local out
  if ! out=$(quint run "$SPEC" "$@" ${SEED_ARGS[@]+"${SEED_ARGS[@]}"} 2>&1); then
    echo "$out"
    exit 1
  fi
  echo "$out" | grep -E '^\[(ok|violation)\]'
}

step "Quint: typecheck and tests"
quint typecheck "$SPEC"
quint test "$SPEC"

step "Quint: simulate 1000 executions, checking invariant 'safety'"
quint_run --invariant=safety --max-samples=1000 --max-steps="$MAX_STEPS"

step "Quint: generate $N_TRACES ITF traces, checking invariant 'safety' in every state"
rm -rf "$TRACES"
mkdir -p "$TRACES"
quint_run --mbt --invariant=safety \
  --max-samples="$N_TRACES" --n-traces="$N_TRACES" --max-steps="$MAX_STEPS" \
  --out-itf="$TRACES/trace_{seq}.itf.json"

step "C++: build"
cmake -S "$ROOT" -B "$BUILD" >/dev/null
cmake --build "$BUILD"

set +e

step "C++: unit tests"
"$BUILD/elevator_tests" --gtest_brief=1
unit_status=$?

step "C++: replay Quint traces (MBT driver)"
MBT_TRACES_DIR="$TRACES" "$BUILD/mbt_driver" --gtest_brief=1 2>&1 | tee "$LOG"
mbt_status=${PIPESTATUS[0]}

set -e

echo
[[ $mbt_status -eq 0 ]] && echo "MBT:  PASS" || echo "MBT:  FAIL (log: $LOG)"
[[ $unit_status -eq 0 ]] && echo "UNIT: PASS" || echo "UNIT: FAIL"
[[ $mbt_status -eq 0 && $unit_status -eq 0 ]]
```

Real output of `./scripts/mbt.sh` on the fixed code. Both Quint steps report the verdict for `safety`:

```
$ ./scripts/mbt.sh

==> Quint: typecheck and tests

  elevator
    ok hallCallLifecycleTest passed 1 test(s)
    ok litButtonIsIgnoredTest passed 1 test(s)
    ok noDepartureWithOpenDoorTest passed 1 test(s)
    ok scanKeepsDirectionTest passed 1 test(s)
    ok closestElevatorIsChosenTest passed 1 test(s)
```

```
  5 passing (66ms)

==> Quint: simulate 1000 executions, checking invariant 'safety'
[ok] No violation found (259ms at 3861 traces/second).

==> Quint: generate 50 ITF traces, checking invariant 'safety' in every state
[ok] No violation found (420ms at 119 traces/second).

==> C++: build
[ 38%] Built target elevator
[ 53%] Built target gtest
[ 69%] Built target gtest_main
[ 84%] Built target elevator_tests
[100%] Built target mbt_driver

==> C++: unit tests
Running main() from .../googletest-src/googletest/src/gtest_main.cc
[==========] 11 tests from 2 test suites ran. (0 ms total)
[  PASSED  ] 11 tests.

==> C++: replay Quint traces (MBT driver)
Running main() from .../googletest-src/googletest/src/gtest_main.cc
[   MBT    ] trace_0.itf.json: 60 steps replayed
[   MBT    ] trace_1.itf.json: 60 steps replayed
[   MBT    ] ...   (46 more traces, each 60 steps replayed)
[   MBT    ] trace_8.itf.json: 60 steps replayed
[   MBT    ] trace_9.itf.json: 60 steps replayed
[   MBT    ] replayed 50 traces, 3000 steps: assignCall=257 closeDoor=222 deliverAny=925 departure=268
openDoor=248 pressCarButton=417 pressHallButton=343 travel=320
[==========] 51 tests from 2 test suites ran. (1115 ms total)
[  PASSED  ] 51 tests.

MBT:  PASS
UNIT: PASS
```

Result	
Quint `run` scenarios	5 / 5 passing
`safety` in 1000 random executions	no violation
`safety` in every state of the 50 generated traces	no violation
C++ unit and scenario tests	11 / 11 passing
MBT traces replayed	50 traces, 3000 steps, 0 divergences
Same loop with the injected bug	23 / 50 traces diverged; 1 / 11 unit tests failed

11. Appendix — Full Quint Spec (elevator.qnt)

spec/elevator.qnt

```
// -*- mode: Bluespec; -*-
/**
 * Elevator group control system.
 *
 * A building has NUM_FLOORS floors and a group of elevators. On every floor
 * there are hall buttons (Up / Down). Inside every car there is a panel with
 * one button per floor (car calls).
 *
 * Components:
 *   - Hall panels: when a passenger presses a hall button, its lamp lights up
 *     and a HallCall message is sent to the dispatcher.
 *   - Dispatcher: collects hall calls and assigns each of them to exactly one
 *     elevator, choosing the one that is (according to its possibly stale
 *     knowledge) closest to the calling floor.
 *   - Elevators: each car runs its own controller. It receives assignments,
 *     moves floor by floor using the SCAN ("elevator") algorithm, opens its
 *     doors at target floors and reports its position and served calls back
 *     to the dispatcher.
 *
 * Communication between the dispatcher and the elevators is asynchronous:
 * messages are kept in a set and may be delivered in any order and with any
 * delay. Messages are never lost.
 *
 * Car calls are handled locally by the car (they never go through the
 * dispatcher), exactly as in real installations.
 */
module elevator {

  // ---------------------------------------------------------------------------
  // Parameters
  // ---------------------------------------------------------------------------

  pure val NUM_FLOORS = 4
  pure val BOTTOM = 0
  pure val TOP = NUM_FLOORS - 1
  pure val FLOORS = BOTTOM.to(TOP)
  pure val ELEVATORS = Set("A", "B")

  // ---------------------------------------------------------------------------
  // Types
  // ---------------------------------------------------------------------------

  type Dir = Up | Down

  // Direction the car (elevator cabin) is committed to. It is set on departure
  // and reset to Idle when the car runs out of work. An Idle car may already
  // have new targets that arrived while it was stopped.
  type Heading = Going(Dir) | Idle

  type Door = Open | Closed

  type Motor = Stopped | Running

  // A request made from a hall panel: "I am on floor `floor` and want to go `dir`".
  type HallCall = { floor: int, dir: Dir }

  type ElevatorState = {
    // Current floor. While running, this is the last floor the car passed.
    floor: int,
    door: Door,
    motor: Motor,
```

```
  heading: Heading,
  // Floors requested from inside the car.
  carCalls: Set[int],
  // Hall calls assigned to this car by the dispatcher and not yet served.
  hallStops: Set[HallCall],
  // Sequence number of the last status report, used to discard stale reports.
  statusSeq: int,
}

// The dispatcher's (possibly outdated) view of an elevator's position.
type PositionReport = { floor: int, seq: int }

type DispatcherState = {
  // Hall calls received but not yet assigned to any elevator.
  pending: Set[HallCall],
  // Hall calls assigned to an elevator and not yet confirmed as served.
  assigned: HallCall -> str,
  // Last known position of every elevator.
  known: str -> PositionReport,
}

type Message =
  // Hall panel -> dispatcher
  | HallCallMsg(HallCall)
  // Dispatcher -> elevator
  | AssignMsg({ elevator: str, call: HallCall })
  // Elevator -> dispatcher
  | StatusMsg({ elevator: str, floor: int, seq: int })
  // Elevator -> dispatcher
  | ServedMsg({ elevator: str, call: HallCall })

// ---------------------------------------------------------------------------
// State variables
// ---------------------------------------------------------------------------

var elevators: str -> ElevatorState
var dispatcher: DispatcherState
// Hall button lamps that are currently lit.
var hallLamps: Set[HallCall]
// Messages in flight.
var network: Set[Message]

// ---------------------------------------------------------------------------
// Pure helpers
// ---------------------------------------------------------------------------

pure def abs(x: int): int = if (x < 0) 0 - x else x

// There is no Down button on the bottom floor and no Up button on the top floor.
pure def isValidHallCall(c: HallCall): bool =
  FLOORS.contains(c.floor)
    and not(c.floor == TOP and c.dir == Up)
    and not(c.floor == BOTTOM and c.dir == Down)

pure val HALL_CALLS: Set[HallCall] =
  FLOORS
    .map(f => Set({ floor: f, dir: Up }, { floor: f, dir: Down }))
    .flatten()
    .filter(isValidHallCall)

// All floors at which the car has to stop.
pure def targets(es: ElevatorState): Set[int] =
  es.carCalls.union(es.hallStops.map(c => c.floor))

// SCAN: keep going in the current direction while there is work ahead,
// otherwise reverse. Only meaningful when the car has a target elsewhere.
pure def nextDir(es: ElevatorState): Dir = {
  val ts = targets(es)
  val above = ts.exists(t => t > es.floor)
  val below = ts.exists(t => t < es.floor)
```

```
  if (es.heading == Going(Up) and above) Up
  else if (es.heading == Going(Down) and below) Down
  else if (above) Up
  else Down
}

pure def delta(h: Heading): int =
  match h {
    | Going(d) => if (d == Up) 1 else -1
    | Idle => 0
  }

// Removes a key from a map.
pure def without(m: HallCall -> str, k: HallCall): HallCall -> str =
  m.keys().exclude(Set(k)).mapBy(x => m.get(x))

// Elevators with the minimal distance (by last known position) to a floor.
pure def closestElevators(d: DispatcherState, floor: int): Set[str] =
  ELEVATORS.filter(e =>
    ELEVATORS.forall(o =>
      abs(d.known.get(e).floor - floor) <= abs(d.known.get(o).floor - floor)
    )
  )

// ---------------------------------------------------------------------------
// Initial state
// ---------------------------------------------------------------------------

action init = all {
  elevators' = ELEVATORS.mapBy(_ => {
    floor: BOTTOM,
    door: Closed,
    motor: Stopped,
    heading: Idle,
    carCalls: Set(),
    hallStops: Set(),
    statusSeq: 0,
  }),
  dispatcher' = {
    pending: Set(),
    assigned: Map(),
    known: ELEVATORS.mapBy(_ => { floor: BOTTOM, seq: 0 }),
  },
  hallLamps' = Set(),
  network' = Set(),
}

// ---------------------------------------------------------------------------
// Environment: passengers
// ---------------------------------------------------------------------------

// A passenger presses a hall button. Pressing an already lit button has no effect.
action pressHallButton(c: HallCall): bool = all {
  HALL_CALLS.contains(c),
  not(hallLamps.contains(c)),
  hallLamps' = hallLamps.union(Set(c)),
  network' = network.union(Set(HallCallMsg(c))),
  elevators' = elevators,
  dispatcher' = dispatcher,
}

// A passenger inside car `e` presses the button for floor `f`.
action pressCarButton(e: str, f: int): bool = {
  val es = elevators.get(e)
  all {
    FLOORS.contains(f),
    f != es.floor,
    not(es.carCalls.contains(f)),
    elevators' = elevators.set(e, { ...es, carCalls: es.carCalls.union(Set(f)) }),
    dispatcher' = dispatcher,
```

```
    hallLamps' = hallLamps,
    network' = network,
  }
}

// ---------------------------------------------------------------------------
// Dispatcher
// ---------------------------------------------------------------------------

action receiveHallCall(c: HallCall): bool = {
  val known = dispatcher.pending.contains(c) or dispatcher.assigned.keys().contains(c)
  all {
    network.contains(HallCallMsg(c)),
    network' = network.exclude(Set(HallCallMsg(c))),
    dispatcher' = if (known) dispatcher
                  else { ...dispatcher, pending: dispatcher.pending.union(Set(c)) },
    elevators' = elevators,
    hallLamps' = hallLamps,
  }
}

// Assigns a pending call to a specific elevator. Only the closest elevators qualify.
action assignTo(c: HallCall, e: str): bool = all {
  dispatcher.pending.contains(c),
  closestElevators(dispatcher, c.floor).contains(e),
  dispatcher' = {
    ...dispatcher,
    pending: dispatcher.pending.exclude(Set(c)),
    assigned: dispatcher.assigned.put(c, e),
  },
  network' = network.union(Set(AssignMsg({ elevator: e, call: c }))),
  elevators' = elevators,
  hallLamps' = hallLamps,
}

// Ties between equally close elevators are resolved non-deterministically.
action assignCall(c: HallCall): bool = {
  nondet target = closestElevators(dispatcher, c.floor).oneOf()
  assignTo(c, target)
}

action receiveStatus(s: { elevator: str, floor: int, seq: int }): bool = {
  val fresh = s.seq > dispatcher.known.get(s.elevator).seq
  all {
    network.contains(StatusMsg(s)),
    network' = network.exclude(Set(StatusMsg(s))),
    dispatcher' = if (fresh)
                    { ...dispatcher,
                      known: dispatcher.known.set(s.elevator, { floor: s.floor, seq: s.seq }) }
                  else dispatcher,
    elevators' = elevators,
    hallLamps' = hallLamps,
  }
}

// The call is confirmed as served: forget it and switch the hall lamp off.
action receiveServed(s: { elevator: str, call: HallCall }): bool = all {
  network.contains(ServedMsg(s)),
  network' = network.exclude(Set(ServedMsg(s))),
  dispatcher' = { ...dispatcher, assigned: without(dispatcher.assigned, s.call) },
  hallLamps' = hallLamps.exclude(Set(s.call)),
  elevators' = elevators,
}

// ---------------------------------------------------------------------------
// Elevator controller
// ---------------------------------------------------------------------------

action receiveAssign(a: { elevator: str, call: HallCall }): bool = {
  val es = elevators.get(a.elevator)
```

```
    all {
      network.contains(AssignMsg(a)),
      network' = network.exclude(Set(AssignMsg(a))),
      elevators' = elevators.set(a.elevator,
                                 { ...es, hallStops: es.hallStops.union(Set(a.call)) }),
      dispatcher' = dispatcher,
      hallLamps' = hallLamps,
    }
  }

  // The car starts moving towards its next target. Work on the current floor
  // must be done first (see openDoor).
  action departure(e: str): bool = {
    val es = elevators.get(e)
    val ts = targets(es)
    all {
      es.motor == Stopped,
      es.door == Closed,
      ts != Set(),
      not(ts.contains(es.floor)),
      elevators' = elevators.set(e, { ...es, motor: Running, heading: Going(nextDir(es)) }),
      dispatcher' = dispatcher,
      hallLamps' = hallLamps,
      network' = network,
    }
  }

  // The car moves by one floor. If the new floor is a target, it stops there.
  // Every floor change is reported to the dispatcher.
  action travel(e: str): bool = {
    val es = elevators.get(e)
    val newFloor = es.floor + delta(es.heading)
    val seq = es.statusSeq + 1
    all {
      es.motor == Running,
      elevators' = elevators.set(e, {
        ...es,
        floor: newFloor,
        motor: if (targets(es).contains(newFloor)) Stopped else Running,
        statusSeq: seq,
      }),
      network' = network.union(Set(StatusMsg({ elevator: e, floor: newFloor, seq: seq }))),
      dispatcher' = dispatcher,
      hallLamps' = hallLamps,
    }
  }

  // The car opens its doors at a target floor. All car calls and hall calls
  // for this floor are served at once.
  action openDoor(e: str): bool = {
    val es = elevators.get(e)
    val served = es.hallStops.filter(c => c.floor == es.floor)
    val remaining = { ...es,
                      carCalls: es.carCalls.exclude(Set(es.floor)),
                      hallStops: es.hallStops.exclude(served) }
    all {
      es.motor == Stopped,
      es.door == Closed,
      targets(es).contains(es.floor),
      elevators' = elevators.set(e, {
        ...remaining,
        door: Open,
        heading: if (targets(remaining) == Set()) Idle else es.heading,
      }),
      network' = network.union(served.map(c => ServedMsg({ elevator: e, call: c }))),
      dispatcher' = dispatcher,
      hallLamps' = hallLamps,
    }
  }
```

```
action closeDoor(e: str): bool = {
  val es = elevators.get(e)
  all {
    es.door == Open,
    elevators' = elevators.set(e, { ...es, door: Closed }),
    dispatcher' = dispatcher,
    hallLamps' = hallLamps,
    network' = network,
  }
}

// ---------------------------------------------------------------------------
// Network
// ---------------------------------------------------------------------------

action deliver(m: Message): bool =
  match m {
    | HallCallMsg(c) => receiveHallCall(c)
    | AssignMsg(a) => receiveAssign(a)
    | StatusMsg(s) => receiveStatus(s)
    | ServedMsg(s) => receiveServed(s)
  }

action deliverAny: bool = all {
  network != Set(),
  nondet m = network.oneOf()
  deliver(m),
}

// ---------------------------------------------------------------------------
// Transition relation
// ---------------------------------------------------------------------------

action step = {
  nondet e = ELEVATORS.oneOf()
  nondet f = FLOORS.oneOf()
  nondet c = HALL_CALLS.oneOf()
  any {
    pressHallButton(c),
    pressCarButton(e, f),
    assignCall(c),
    departure(e),
    travel(e),
    openDoor(e),
    closeDoor(e),
    deliverAny,
  }
}

// ---------------------------------------------------------------------------
// Safety invariants
// ---------------------------------------------------------------------------

// A car never moves with open doors.
val doorsClosedWhileRunning =
  ELEVATORS.forall(e => elevators.get(e).motor == Running implies elevators.get(e).door == Closed)

// A car never leaves the shaft.
val staysInShaft =
  ELEVATORS.forall(e => FLOORS.contains(elevators.get(e).floor))

// A running car always has a direction.
val runningHasHeading =
  ELEVATORS.forall(e => elevators.get(e).motor == Running implies elevators.get(e).heading != Idle)

// A running car always has a target ahead of it, so it never runs aimlessly.
val runningHasTargetAhead =
  ELEVATORS.forall(e => {
    val es = elevators.get(e)
    es.motor == Running implies targets(es).exists(t => (t - es.floor) * delta(es.heading) > 0)
```

```
  })

// Every hall call is owned by at most one elevator (in flight or accepted).
val noDoubleService =
  HALL_CALLS.forall(c => {
    val owners = ELEVATORS.filter(e =>
      elevators.get(e).hallStops.contains(c)
        or network.contains(AssignMsg({ elevator: e, call: c }))
    )
    owners.size() <= 1
  })

// The dispatcher's assignment always matches the elevator that owns the call.
val assignmentConsistent =
  ELEVATORS.forall(e =>
    elevators.get(e).hallStops.forall(c =>
      dispatcher.assigned.keys().contains(c) and dispatcher.assigned.get(c) == e
    )
  )

// No lost calls: every lit hall lamp is tracked by exactly one stage of the
// pipeline (message to the dispatcher, pending, or assigned).
val noLostHallCall =
  hallLamps.forall(c => {
    val inFlight = if (network.contains(HallCallMsg(c))) 1 else 0
    val pending = if (dispatcher.pending.contains(c)) 1 else 0
    val assigned = if (dispatcher.assigned.keys().contains(c)) 1 else 0
    inFlight + pending + assigned == 1
  })

// Every assigned call is on its way to, held by, or confirmed by its elevator.
val assignedCallIsTracked =
  dispatcher.assigned.keys().forall(c => {
    val e = dispatcher.assigned.get(c)
    network.contains(AssignMsg({ elevator: e, call: c }))
      or elevators.get(e).hallStops.contains(c)
      or network.contains(ServedMsg({ elevator: e, call: c }))
  })

// A lamp is switched off only after the call has been served.
val noTrackingWithoutLamp =
  dispatcher.pending.union(dispatcher.assigned.keys()).forall(c => hallLamps.contains(c))

val safety = and {
  doorsClosedWhileRunning,
  staysInShaft,
  runningHasHeading,
  runningHasTargetAhead,
  noDoubleService,
  assignmentConsistent,
  noLostHallCall,
  assignedCallIsTracked,
  noTrackingWithoutLamp,
}

// ---------------------------------------------------------------------------
// Witnesses: these are expected to be violated. Running
//   quint run elevator.qnt --invariant=<witness>
// produces an example trace that reaches the interesting situation.
// ---------------------------------------------------------------------------

// Both cars are moving at the same time.
val witnessBothRunning =
  not(ELEVATORS.forall(e => elevators.get(e).motor == Running))

// Some elevator has served a hall call and is reporting it to the dispatcher.
val witnessHallCallServed =
  not(network.exists(m => match m { | ServedMsg(_) => true | _ => false }))

// The dispatcher acts on a stale position of some elevator.
```

```
  val witnessStaleKnowledge =
    ELEVATORS.forall(e => dispatcher.known.get(e).floor == elevators.get(e).floor)

  // -----------------------------------------------------------------------------
  // Tests
  // -----------------------------------------------------------------------------

  pure val call2Down = { floor: 2, dir: Down }

  // A single hall call travels through the whole pipeline.
  run hallCallLifecycleTest =
    init
      .then(pressHallButton(call2Down))
      .then(deliver(HallCallMsg(call2Down)))
      .expect(dispatcher.pending == Set(call2Down))
      .then(assignTo(call2Down, "A"))
      .then(deliver(AssignMsg({ elevator: "A", call: call2Down })))
      .then(departure("A"))
      .expect(elevators.get("A").heading == Going(Up))
      .then(travel("A"))
      .expect(elevators.get("A").motor == Running)
      .then(travel("A"))
      .expect(elevators.get("A").floor == 2 and elevators.get("A").motor == Stopped)
      .then(openDoor("A"))
      .expect(elevators.get("A").door == Open and elevators.get("A").heading == Idle)
      .then(deliver(ServedMsg({ elevator: "A", call: call2Down })))
      .expect(hallLamps == Set() and dispatcher.assigned == Map())
      .expect(safety)

  // Pressing a lit hall button does nothing.
  run litButtonIsIgnoredTest =
    init
      .then(pressHallButton(call2Down))
      .then(pressHallButton(call2Down))
      .fail()

  // The car cannot leave while its doors are open.
  run noDepartureWithOpenDoorTest =
    init
      .then(pressCarButton("A", 3))
      .then(departure("A"))
      .then(travel("A"))
      .then(pressCarButton("A", 0))
      .then(travel("A"))
      .then(travel("A"))
      .expect(elevators.get("A").floor == 3)
      .then(openDoor("A"))
      .then(departure("A"))
      .fail()

  // SCAN: a car going up serves the upper floor before reversing.
  run scanKeepsDirectionTest =
    init
      .then(pressCarButton("A", 2))
      .then(departure("A"))
      .then(travel("A"))
      .then(pressCarButton("A", 0))
      .then(travel("A"))
      .then(openDoor("A"))
      .then(closeDoor("A"))
      .then(departure("A"))
      .expect(elevators.get("A").heading == Going(Down))
      .expect(safety)

  // A hall call is assigned to the closest elevator according to the dispatcher.
  run closestElevatorIsChosenTest =
    init
      .then(pressCarButton("B", 3))
      .then(departure("B"))
      .then(travel("B"))
```

```
      .then(deliver(StatusMsg({ elevator: "B", floor: 1, seq: 1 })))
      .then(pressHallButton({ floor: 2, dir: Up }))
      .then(deliver(HallCallMsg({ floor: 2, dir: Up })))
      .then(assignTo({ floor: 2, dir: Up }, "A"))
      .fail()
}
```